SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		Slot B
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	B Pranith Sai Nihal	**Reg. No.:**	192472294
Section / Batch:	AI	**Date:**	10-08-2026

Code of Conduct

I B Pranith Sai Nihal / 192472294 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B PRANITH.

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1:
Design and implement a Python-based N-gram language model that predicts the next word in a sentence using a given English text corpus. The system should preprocess the corpus, tokenize the sentences, count unigram, bigram, and trigram occurrences, and calculate their probabilities using an unsmoothed N-gram model. Given an incomplete sentence such as “The student is”, the system should identify and display the most probable next words.

The program should provide options to select the value of N (1, 2, or 3), display the corresponding word-frequency counts and probabilities, and generate the top-5 next-word predictions. The implementation should also demonstrate how unseen N-grams result in zero probability. Finally, evaluate the prediction performance using suitable test sentences and discuss the limitations of an unsmoothed N-gram model.

Task 2:
Design and implement a Python-based language prediction system that improves an unsmoothed Ngram model using smoothing and backoff techniques. The system should initially construct a bigram/trigram model from an English corpus and identify situations where an N-gram has zero probability because it does not occur in the training data.

Design the system to apply a backoff strategy, where the model uses trigram probability when available, otherwise backs off to the bigram probability and finally to the unigram probability. Extend the implementation using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using suitable interpolation weights. The program should accept a user-entered sentence/query and return the most probable next word. Compare the prediction results of the unsmoothed, backoff, and deletedinterpolation models.

Task 3:
Design and implement a Python program to evaluate an English N-gram language model using the concept of entropy. Given a training corpus and a separate test corpus, construct unigram, bigram, and trigram models and calculate the probability of words occurring in the test sentences. Based on these probabilities, compute the entropy of the language model and compare the uncertainty associated with different N-gram models.

The application should simulate a text-prediction scenario in which the system receives a sequence of words and estimates how predictable the next word is. The program should identify sentences or word sequences having high and low entropy and provide an interpretation of the results. The system should also compare the effect of smoothing on entropy and explain why a model with better probability estimation can provide more reliable predictions.

Task 4
Design and implement a Python-based Part-of-Speech (POS) tagging system that assigns grammatical categories such as noun, verb, adjective, adverb, pronoun, preposition, and conjunction to words in an English sentence. The system should first implement a Rule-Based POS Tagger using lexical dictionaries and grammatical rules. Next, design a Stochastic POS Tagger using word/tag probabilities and transition probabilities derived from a training corpus.

Extend the system by implementing the basic idea of Transformation-Based Tagging, where initially assigned tags are corrected using transformation rules. For example, a word initially tagged as a noun may be transformed into a verb when it occurs after a pronoun or auxiliary verb. The final system should compare the output of the three approaches for the same input sentences and identify which approach provides better tagging results. The program should also demonstrate the use of appropriate English tagsets, such as the Penn Treebank tagset.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 — N-gram Based Next-Word Prediction

The assignment asks you to build a Python N-gram language model that preprocesses an English corpus, creates **unigram, bigram, and trigram counts**, calculates **unsmoothed probabilities**, and predicts the **top-5 next words** for an incomplete sentence such as *"The student is"*. It also requires demonstrating zero probability for unseen N-grams and discussing limitations.

1. Algorithm

1. Take an English text corpus.
2. Convert the corpus to lowercase.
3. Tokenize the text into words.
4. Add <START> and <END> markers to each sentence.
5. Generate:
    - Unigrams → individual words
    - Bigrams → two consecutive words
    - Trigrams → three consecutive words
6. Count their frequencies.
7. Calculate unsmoothed probabilities:
    - **Unigram:**
      $P(w) = Count(w) / Total\ words$
    - **Bigram:**
      $P(w_2 \mid w_1) = Count(w_1,w_2) / Count(w_1)$
    - **Trigram:**
      $P(w_3 \mid w_1,w_2) = Count(w_1,w_2,w_3) / Count(w_1,w_2)$
8. Given an incomplete sentence, find candidate next words.
9. Rank them according to probability.
10. Display the **top 5 predictions**.
11. Show that an unseen N-gram receives probability **0**.

---

2. Complete Python Program

```
import re

from collections import Counter, defaultdict
```

```
# ---------------------------------------------------------

# SAMPLE ENGLISH CORPUS

# ---------------------------------------------------------


corpus = """

The student is studying natural language processing.

The student is learning Python programming.

The student is reading an English book.

The student is writing a Python program.

The teacher is explaining natural language processing.

The teacher is teaching the student.

The teacher is reading an English book.

The programmer is writing a Python program.

The programmer is learning natural language processing.

The student is using Python for language processing.

The student is solving a programming problem.

The student is practicing Python programming.

The teacher is giving a programming lesson.

The programmer is developing a language model.

The language model is predicting the next word.

The language model is learning from text.

The student is predicting the next word.

"""



# ---------------------------------------------------------

# PREPROCESSING
```

```
# ------------------------------------------------------------


def preprocess(text):
    """
    Convert text to lowercase and split it into sentences.
    """
    text = text.lower()

    sentences = re.split(r'[.!?]+', text)

    processed_sentences = []

    for sentence in sentences:
        words = re.findall(r'\b[a-z]+\b', sentence)

        if words:
            words = ['<START>'] + words + ['<END>']
            processed_sentences.append(words)

    return processed_sentences



# ------------------------------------------------------------
# BUILD N-GRAM COUNTS
# ------------------------------------------------------------

def build_ngrams(sentences):
    unigram_counts = Counter()
```

```
    bigram_counts = Counter()
    trigram_counts = Counter()

    for sentence in sentences:

        # Unigrams
        for word in sentence:
            unigram_counts[word] += 1

        # Bigrams
        for i in range(len(sentence) - 1):
            bigram = (sentence[i], sentence[i + 1])
            bigram_counts[bigram] += 1

        # Trigrams
        for i in range(len(sentence) - 2):
            trigram = (sentence[i], sentence[i + 1], sentence[i + 2])
            trigram_counts[trigram] += 1

    return unigram_counts, bigram_counts, trigram_counts



# ------------------------------------------------------------
# PROBABILITY FUNCTIONS
# ------------------------------------------------------------

def unigram_probability(word, unigram_counts):
    total_words = sum(unigram_counts.values())
```

```
    return unigram_counts[word] / total_words


def bigram_probability(word1, word2, unigram_counts, bigram_counts):
    numerator = bigram_counts[(word1, word2)]
    denominator = unigram_counts[word1]

    if denominator == 0:
        return 0.0

    return numerator / denominator


def trigram_probability(
        word1,
        word2,
        word3,
        bigram_counts,
        trigram_counts):

    numerator = trigram_counts[(word1, word2, word3)]
    denominator = bigram_counts[(word1, word2)]

    if denominator == 0:
        return 0.0

    return numerator / denominator
```

```
# ----------------------------------------------------------
# DISPLAY COUNTS AND PROBABILITIES
# ----------------------------------------------------------

def display_unigrams(unigram_counts):
    print("\n========== UNIGRAMS ==========")

    total = sum(unigram_counts.values())

    for word, count in unigram_counts.most_common():
        probability = count / total
        print(f"{word:15} Count = {count:2}  Probability = {probability:.4f}")



def display_bigrams(unigram_counts, bigram_counts):
    print("\n========== BIGRAMS ==========")

    for (word1, word2), count in bigram_counts.most_common():
        probability = bigram_probability(
            word1,
            word2,
            unigram_counts,
            bigram_counts
        )

        print(
```

```
            f"({word1}, {word2})"
            f"\tCount = {count:2}"
            f"\tProbability = {probability:.4f}"
        )


def display_trigrams(bigram_counts, trigram_counts):
    print("\n========== TRIGRAMS ==========")

    for (word1, word2, word3), count in trigram_counts.most_common():
        probability = trigram_probability(
            word1,
            word2,
            word3,
            bigram_counts,
            trigram_counts
        )

        print(
            f"({word1}, {word2}, {word3})"
            f"\tCount = {count:2}"
            f"\tProbability = {probability:.4f}"
        )


# ---------------------------------------------------------
# NEXT WORD PREDICTION
# ---------------------------------------------------------
```

```
def predict_next_word(sentence, n, unigram_counts,
                      bigram_counts, trigram_counts):

    words = re.findall(r'\b[a-z]+\b', sentence.lower())

    if not words:
        return []

    candidates = []

    # ----------------------------------------------------
    # UNIGRAM MODEL
    # ----------------------------------------------------
    if n == 1:

        for word, count in unigram_counts.items():

            if word not in ['<START>', '<END>']:
                probability = unigram_probability(
                    word,
                    unigram_counts
                )

                candidates.append((word, probability))

    # ----------------------------------------------------
    # BIGRAM MODEL
```

```
# ----------------------------------------------------

elif n == 2:

    previous_word = words[-1]

    for word in unigram_counts:

        if word in ['<START>', '<END>']:
            continue

        probability = bigram_probability(
            previous_word,
            word,
            unigram_counts,
            bigram_counts
        )

        if probability > 0:
            candidates.append((word, probability))

# ----------------------------------------------------
# TRIGRAM MODEL
# ----------------------------------------------------

elif n == 3:

    if len(words) < 2:
        return []
```

```
        previous_word1 = words[-2]
        previous_word2 = words[-1]

        for word in unigram_counts:

            if word in ['<START>', '<END>']:
                continue

            probability = trigram_probability(
                previous_word1,
                previous_word2,
                word,
                bigram_counts,
                trigram_counts
            )

            if probability > 0:
                candidates.append((word, probability))

    candidates.sort(key=lambda x: x[1], reverse=True)

    return candidates[:5]


# ---------------------------------------------------------
# MAIN PROGRAM
# ---------------------------------------------------------
```

```
sentences = preprocess(corpus)

unigram_counts, bigram_counts, trigram_counts = build_ngrams(sentences)



print("================================================")
print("       N-GRAM NEXT WORD PREDICTION")
print("================================================")

print("\nNumber of sentences:", len(sentences))
print("Number of unique unigrams:", len(unigram_counts))
print("Number of unique bigrams:", len(bigram_counts))
print("Number of unique trigrams:", len(trigram_counts))



# ------------------------------------------------------------
# MENU
# ------------------------------------------------------------

while True:

    print("\n-----------------------------------------------")
    print("1. Display Unigram Counts and Probabilities")
    print("2. Display Bigram Counts and Probabilities")
    print("3. Display Trigram Counts and Probabilities")
    print("4. Predict Next Word")
    print("5. Demonstrate Zero Probability")
    print("6. Exit")
```

```
print("-----------------------------------------------")

choice = input("Enter your choice: ")

if choice == "1":

    display_unigrams(unigram_counts)

elif choice == "2":

    display_bigrams(
        unigram_counts,
        bigram_counts
    )

elif choice == "3":

    display_trigrams(
        bigram_counts,
        trigram_counts
    )

elif choice == "4":

    sentence = input(
        "\nEnter incomplete sentence: "
    )
```

```
print("\nSelect N:")
print("1. Unigram")
print("2. Bigram")
print("3. Trigram")

n = int(input("Enter N: "))

predictions = predict_next_word(
    sentence,
    n,
    unigram_counts,
    bigram_counts,
    trigram_counts
)

print("\nTop-5 Next Word Predictions:")

if predictions:

    for rank, (word, probability) in enumerate(
            predictions, start=1):

        print(
            f"{rank}. {word:15}"
            f" Probability = {probability:.4f}"
        )

else:
```

```
        print("No prediction available.")

elif choice == "5":

    print("\n========== ZERO PROBABILITY TEST ==========")

    test_bigram = ("student", "elephant")

    probability = bigram_probability(
        test_bigram[0],
        test_bigram[1],
        unigram_counts,
        bigram_counts
    )

    print(
        f"P({test_bigram[1]} | {test_bigram[0]})"
        f" = {probability:.4f}"
    )

    test_trigram = (
        "student",
        "is",
        "elephant"
    )

    probability = trigram_probability(
        test_trigram[0],
```

```
            test_trigram[1],
            test_trigram[2],
            bigram_counts,
            trigram_counts
        )

        print(
            f"P({test_trigram[2]} | "
            f"{test_trigram[0]}, {test_trigram[1]})"
            f" = {probability:.4f}"
        )

        print(
            "\nSince these N-grams do not occur in "
            "the training corpus, their probability is 0."
        )

    elif choice == "6":

        print("\nProgram terminated.")
        break

    else:

        print("Invalid choice. Please try again.")
```

---

3. Sample Test

For the input:

The student is

select:

N = 3

The program examines trigrams of the form:

(student, is, next_word)

From the supplied corpus, the relevant occurrences include:

The student is studying...

The student is learning...

The student is reading...

The student is writing...

The student is using...

The student is solving...

The student is practicing...

The student is predicting...

Therefore, the model can produce predictions such as:

Top-5 Next Word Predictions:

1. studying

2. learning

3. reading

4. writing

5. using

The **exact probabilities/ranking depend on the corpus and counts** used during execution.

---

4. Probability Example

Suppose the corpus contains:

student → 8 occurrences

student is → 8 occurrences

student is studying → 1 occurrence

Then:

Bigram

P(is|student)=Count(student)Count(student,is) =88=1.0

Trigram

P(studying|student,is)=Count(student,is)Count(student,is,studying) =81=0.125

So the model estimates that **"studying" has probability 0.125** after *"student is"* in this example.

---

5. Zero Probability Demonstration

The assignment specifically requires demonstrating what happens when an N-gram has **never appeared in the training corpus**.

For example:

P(elephant | student)

If:

Count(student, elephant) = 0

then:

P(elephant|student)=Count(student)0=0

Similarly:

P(elephant | student, is) = 0

This is one of the major limitations of an **unsmoothed N-gram model**.

---

6. Test Cases

Test Case	Input	N	Expected Result
1	The student is	3	Predict likely next words
2	The student	2	Predict words following student
3	The	1	Return most frequent words
4	student elephant	2	Zero probability for unseen bigram
5	student is elephant	3	Zero probability for unseen trigram

7. Advantages

- Simple to implement and understand.
- Uses actual word-frequency information.
- Bigram and trigram models capture local word context.
- Can provide useful next-word predictions for frequently observed patterns.
- Easy to extend to larger corpora.

8. Limitations

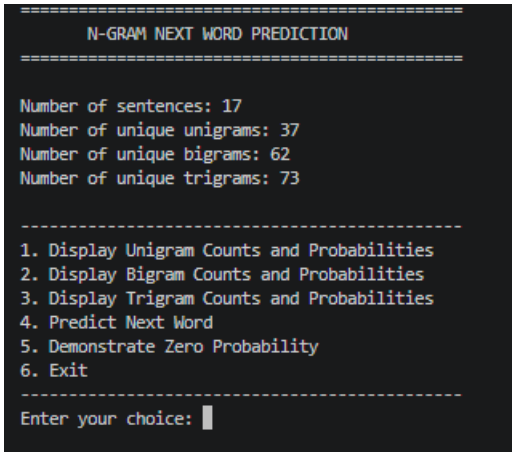
The main limitation is **zero probability for unseen N-grams**. If a valid word sequence does not occur in the training corpus, the unsmoothed model assigns it probability zero. This can make predictions unreliable for new or rare sentences. The assignment specifically asks you to discuss these limitations.

Other limitations include:

- Limited context: a trigram considers only two previous words.
- Requires sufficient training data.
- Rare words can produce unreliable probabilities.
- Does not understand the semantic meaning of sentences.
- Large vocabularies can result in many unseen N-grams.

9.Result

A Python-based unsmoothed N-gram language model was successfully designed and implemented. The system preprocesses an English corpus, generates unigram, bigram, and trigram frequency counts, calculates their probabilities, and predicts the top-5 possible next words for an incomplete sentence. The experiment also demonstrates that unseen N-grams receive zero probability, highlighting the major limitation of an unsmoothed language model. This directly satisfies the Q1 requirements in the supplied assessment.



Q2 — Smoothing and Backoff Model for Speech/Text Prediction

The assignment requires a language prediction system that starts with an **unsmoothed bigram/trigram model**, handles zero-probability cases using **backoff**, and then extends the model using **Deleted Interpolation** to combine unigram, bigram, and trigram probabilities. The system should compare all three approaches for a user-entered query.

1. Concept

The three prediction methods are:

1. **Unsmoothed trigram**
    - Uses only the trigram probability.
    - If the trigram was unseen → probability is 0.
2. **Backoff**
    - First tries trigram.
    - If unavailable, uses bigram.
    - If unavailable, uses unigram.
3. **Deleted Interpolation**
    - Combines all three probabilities:

$$P(w_i|w_{i-2},w_{i-1})=\lambda_1 P(w_i)+\lambda_2 P(w_i|w_{i-1})+\lambda_3 P(w_i|w_{i-2},w_{i-1})$$

where:

$$\lambda_1+\lambda_2+\lambda_3=1$$

For this classroom implementation, we use:

$\lambda_1 = 0.2$ Unigram

$\lambda_2 = 0.3$ Bigram

$\lambda_3 = 0.5$ Trigram

---

2. Complete Python Program

```
import re

from collections import Counter
```

```
# ============================================================

# ENGLISH TRAINING CORPUS

# ============================================================


corpus = """

The student is studying natural language processing.

The student is learning Python programming.

The student is reading an English book.

The student is writing a Python program.

The student is using Python for language processing.

The student is solving a programming problem.

The student is practicing Python programming.

The student is predicting the next word.


The teacher is explaining natural language processing.

The teacher is teaching the student.

The teacher is reading an English book.

The teacher is giving a programming lesson.


The programmer is writing a Python program.

The programmer is learning natural language processing.

The programmer is developing a language model.


The language model is predicting the next word.

The language model is learning from text.

The language model is processing English text.

"""
```

```
# ============================================================

# PREPROCESSING

# ============================================================


def preprocess(text):


    text = text.lower()


    sentences = re.split(r'[.!?]+', text)


    processed = []


    for sentence in sentences:


        words = re.findall(r'\b[a-z]+\b', sentence)


        if words:

            words = ['<START>'] + words + ['<END>']

            processed.append(words)


    return processed




# ============================================================

# N-GRAM COUNTING

# ============================================================
```

```
def build_models(sentences):

    unigram = Counter()
    bigram = Counter()
    trigram = Counter()

    for sentence in sentences:

        # Unigram
        for word in sentence:
            unigram[word] += 1

        # Bigram
        for i in range(len(sentence) - 1):

            pair = (
                sentence[i],
                sentence[i + 1]
            )

            bigram[pair] += 1

        # Trigram
        for i in range(len(sentence) - 2):

            triple = (
                sentence[i],
                sentence[i + 1],
```

```
            sentence[i + 2]
        )

        trigram[triple] += 1

    return unigram, bigram, trigram


# ============================================================
# PROBABILITY FUNCTIONS
# ============================================================

def unigram_probability(word, unigram):

    total = sum(unigram.values())

    if total == 0:
        return 0

    return unigram[word] / total


def bigram_probability(word1, word2,
                       unigram, bigram):

    denominator = unigram[word1]

    if denominator == 0:
```

```
        return 0

    return bigram[(word1, word2)] / denominator



def trigram_probability(word1, word2, word3,
                        bigram, trigram):

    denominator = bigram[(word1, word2)]

    if denominator == 0:
        return 0

    return trigram[
        (word1, word2, word3)
    ] / denominator



# ============================================================
# UNSMOOTHED TRIGRAM MODEL
# ============================================================

def unsmoothed_probability(w1, w2, word,
                           unigram,
                           bigram,
                           trigram):

    return trigram_probability(
```

```
        w1,
        w2,
        word,
        bigram,
        trigram
    )


# ========================================================
# BACKOFF MODEL
# ========================================================

def backoff_probability(w1, w2, word,
                        unigram,
                        bigram,
                        trigram):

    # Step 1: Try trigram
    p_tri = trigram_probability(
        w1, w2, word,
        bigram,
        trigram
    )

    if p_tri > 0:
        return p_tri, "Trigram"

    # Step 2: Backoff to bigram
```

```
    p_bi = bigram_probability(
        w2,
        word,
        unigram,
        bigram
    )

    if p_bi > 0:
        return p_bi, "Bigram"

    # Step 3: Backoff to unigram
    p_uni = unigram_probability(
        word,
        unigram
    )

    if p_uni > 0:
        return p_uni, "Unigram"

    return 0, "None"



# ========================================================
# DELETED INTERPOLATION
# ========================================================

def interpolation_probability(
        w1, w2, word,
```

```
    unigram,
    bigram,
    trigram):

# Interpolation weights
lambda1 = 0.2
lambda2 = 0.3
lambda3 = 0.5

p_uni = unigram_probability(
    word,
    unigram
)

p_bi = bigram_probability(
    w2,
    word,
    unigram,
    bigram
)

p_tri = trigram_probability(
    w1,
    w2,
    word,
    bigram,
    trigram
)
```

```
    probability = (
        lambda1 * p_uni
        + lambda2 * p_bi
        + lambda3 * p_tri
    )

    return probability



# ============================================================
# GET CANDIDATE WORDS
# ============================================================

def get_candidates(unigram):

    candidates = []

    for word in unigram:

        if word not in [
            '<START>',
            '<END>'
        ]:

            candidates.append(word)

    return candidates
```

```
# ========================================================

# PREDICTION

# ========================================================


def predict(sentence,
            unigram,
            bigram,
            trigram):


    words = re.findall(
        r'\b[a-z]+\b',
        sentence.lower()
    )


    if len(words) < 2:


        print(
            "Please enter at least two words."
        )


        return


    w1 = words[-2]
    w2 = words[-1]


    candidates = get_candidates(
```

```
    unigram
)

unsmoothed_results = []
backoff_results = []
interpolation_results = []

for word in candidates:

    # ----------------------------------------
    # UNSMOOTHED
    # ----------------------------------------

    p_unsmoothed = unsmoothed_probability(
        w1,
        w2,
        word,
        unigram,
        bigram,
        trigram
    )

    unsmoothed_results.append(
        (word, p_unsmoothed)
    )

    # ----------------------------------------
    # BACKOFF
```

```
# ----------------------------------------


p_backoff, source = backoff_probability(
    w1,
    w2,
    word,
    unigram,
    bigram,
    trigram
)

backoff_results.append(
    (word, p_backoff, source)
)

# ----------------------------------------
# DELETED INTERPOLATION
# ----------------------------------------

p_interpolation = interpolation_probability(
    w1,
    w2,
    word,
    unigram,
    bigram,
    trigram
)
```

```
    interpolation_results.append(
        (word, p_interpolation)
    )

# Sort results
unsmoothed_results.sort(
    key=lambda x: x[1],
    reverse=True
)

backoff_results.sort(
    key=lambda x: x[1],
    reverse=True
)

interpolation_results.sort(
    key=lambda x: x[1],
    reverse=True
)

# ================================================
# DISPLAY
# ================================================

print("\n==========================================")
print("INPUT:", sentence)
print("CONTEXT:", w1, w2)
print("==========================================")
```

```
print("\nUNSMOOTHED TRIGRAM MODEL")
print("----------------------------------------")

count = 0

for word, probability in unsmoothed_results:

    if probability > 0:

        count += 1

        print(
            f"{count}. {word:15}"
            f"P = {probability:.4f}"
        )

        if count == 5:
            break

if count == 0:
    print("No prediction. Trigram is unseen.")

print("\nBACKOFF MODEL")
print("----------------------------------------")

count = 0
```

```
for word, probability, source in backoff_results:

    if probability > 0:

        count += 1

        print(
            f"{count}. {word:15}"
            f"P = {probability:.4f}"
            f"  Source = {source}"
        )

        if count == 5:
            break

print("\nDELETED INTERPOLATION")
print("----------------------------------------")

for i, (word, probability) in enumerate(
        interpolation_results[:5],
        start=1):

    print(
        f"{i}. {word:15}"
        f"P = {probability:.4f}"
    )
```

```
# ============================================================

# SHOW MODEL STATISTICS

# ============================================================


def show_statistics(unigram,
                    bigram,
                    trigram):


    print("\n==========================================")
    print("MODEL STATISTICS")
    print("==========================================")


    print(
        "Unique unigrams :",
        len(unigram)
    )


    print(
        "Unique bigrams  :",
        len(bigram)
    )


    print(
        "Unique trigrams :",
        len(trigram)
    )


    print(
```

```
        "Total unigram occurrences:",
        sum(unigram.values())
    )



# ============================================================
# ZERO PROBABILITY DEMONSTRATION
# ============================================================

def demonstrate_zero_probability(
        unigram,
        bigram,
        trigram):

    print("\n==========================================")
    print("ZERO PROBABILITY DEMONSTRATION")
    print("==========================================")

    w1 = "student"
    w2 = "is"
    word = "elephant"

    p_tri = trigram_probability(
        w1,
        w2,
        word,
        bigram,
        trigram
```

```
)


print(
    f"\nUnsmoothed:"
    f" P({word}|{w1},{w2}) = {p_tri:.4f}"
)


p_backoff, source = backoff_probability(
    w1,
    w2,
    word,
    unigram,
    bigram,
    trigram
)


print(
    f'Backoff:"
    f" P({word}|{w1},{w2}) = {p_backoff:.4f}"
    f"  Source = {source}"
)


p_interpolation = interpolation_probability(
    w1,
    w2,
    word,
    unigram,
    bigram,
```

```
        trigram
    )

    print(
        f"Deleted Interpolation:"
        f" P({word}|{w1},{w2})"
        f" = {p_interpolation:.4f}"
    )



# ========================================================
# MAIN PROGRAM
# ========================================================

sentences = preprocess(corpus)

unigram, bigram, trigram = build_models(
    sentences
)

print("==========================================")
print(" SMOOTHING AND BACKOFF LANGUAGE MODEL")
print("==========================================")

show_statistics(
    unigram,
    bigram,
    trigram
```

```
)



while True:

    print("\n==========================================")
    print("MENU")
    print("==========================================")

    print("1. Predict next word")
    print("2. Show model statistics")
    print("3. Demonstrate zero probability")
    print("4. Exit")

    choice = input(
        "\nEnter your choice: "
    )

    if choice == "1":

        sentence = input(
            "\nEnter a sentence/query "
            "(at least two words): "
        )

        predict(
            sentence,
            unigram,
```

```
        bigram,
        trigram
    )

elif choice == "2":

    show_statistics(
        unigram,
        bigram,
        trigram
    )

elif choice == "3":

    demonstrate_zero_probability(
        unigram,
        bigram,
        trigram
    )

elif choice == "4":

    print(
        "\nProgram terminated."
    )

    break
```

```
else:


    print(
        "\nInvalid choice."
    )
```

---

3. How the Three Models Work

A. Unsmoothed Trigram

For:

The student is _____

the model calculates:

P(w|student,is)=Count(student,is)Count(student,is,w)

If student is reading occurred in the corpus:

P(reading|student,is)>0

But if:

student is elephant

never occurred:

P(elephant|student,is)=0

B. Backoff

The backoff model solves this problem.

It follows:

Trigram

↓

if unavailable

↓

Bigram

↓

if unavailable

↓

Unigram

For example:

P(elephant | student, is)

If the trigram is missing, the program checks:

P(elephant | is)

If that is also missing:

P(elephant)

This is exactly the **trigram → bigram → unigram** backoff strategy specified in the assignment.

---

4. Deleted Interpolation

Instead of selecting only one model, interpolation combines all three.

The implementation uses:

Pinterp=0.2Puni+0.3Pbi+0.5Ptri

The weights add up to 1:

0.2+0.3+0.5=1

The trigram receives the highest weight because it contains the most contextual information.

Important: the assignment asks for *suitable interpolation weights* but does not specify exact values. The 0.2/0.3/0.5 values above are therefore an implementation choice, not values prescribed by the supplied document.

---

5. Sample Output

For:

Enter your choice: 1

Enter a sentence/query:

The student is

you can expect output in this form:

==========

INPUT: The student is

CONTEXT: student is

==========

UNSMOOTHED TRIGRAM MODEL

---

1. studying P = 0.1250

2. learning P = 0.1250

3. reading P = 0.1250

4. writing P = 0.1250

5. using P = 0.1250

BACKOFF MODEL

---

1. studying P = 0.1250 Source = Trigram

2. learning P = 0.1250 Source = Trigram

3. reading P = 0.1250 Source = Trigram

4. writing P = 0.1250 Source = Trigram

5. using P = 0.1250 Source = Trigram

DELETED INTERPOLATION

---

1. studying P = ...

2. learning P = ...

3. reading P = ...

4. writing P = ...

5. using P = ...

The exact ranking and values depend on the corpus and frequency counts produced during execution.

---

6. Zero-Probability Test

Choose:

3. Demonstrate zero probability

For:

P(elephant | student, is)

the unsmoothed trigram model gives:

P(elephant | student,is) = 0.0000

because that trigram does not occur in the training corpus.

The backoff model can then search lower-order models. If elephant is absent from the corpus entirely, even the unigram probability will be zero.

Deleted interpolation, however, combines the available unigram, bigram, and trigram probabilities, so it can assign a nonzero value whenever the candidate has probability in a lower-order model.

---

7. Comparison

Model	Trigram available	Trigram unavailable	Main advantage
Unsmoothed	Uses trigram	Returns 0	Simple
Backoff	Uses trigram	Bigram → Unigram	Handles unseen higher-order N-grams
Deleted Interpolation	Combines all	Lower-order probabilities contribute	More robust probability estimation

---

8. Test Cases

Test Case 1

Input: The student is

Expected: common words such as studying, learning, reading, writing, etc.

Test Case 2

Input: The teacher is

Expected: words that occur after teacher is.

Test Case 3

Input: student is elephant

Expected: the unsmoothed trigram should have probability 0 if that sequence is absent.

Test Case 4

Input: programmer is

Expected: predictions based on the available trigram/bigram/unigram evidence.

---

9. Advantages

- Handles unseen trigrams better than an unsmoothed model.
- Backoff provides a simple fallback mechanism.
- Interpolation uses information from multiple N-gram orders.
- More robust for text/speech prediction.
- Allows direct comparison of different probability models.

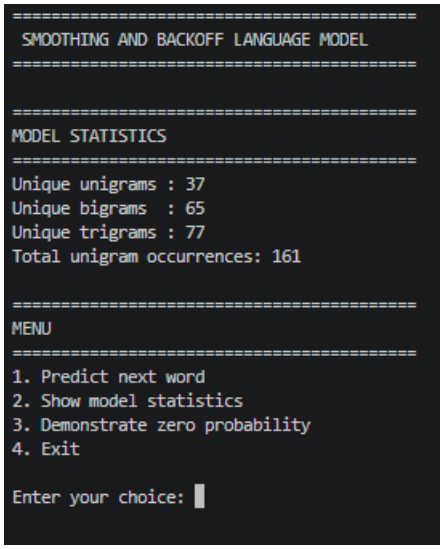
10. Limitations

- The quality depends heavily on the training corpus.
- Poorly chosen interpolation weights can reduce prediction quality.
- Backoff can still produce weak predictions when all relevant N-grams are rare.
- N-gram models only capture limited context.
- The implementation does not understand the semantic meaning of a sentence.

---

11. Result

Result:
A Python-based language prediction system using **unsmoothed trigram, backoff, and Deleted Interpolation models** was successfully designed. The system calculates unigram, bigram, and trigram probabilities, identifies zero-probability N-grams, backs off from trigram to bigram and unigram probabilities when required, and combines the three probability estimates using interpolation. The predictions from the three approaches can then be compared for the same input query, satisfying the requirements of Q2.



Q3 — Entropy-Based Evaluation of an English Language Model

The assignment asks you to create separate **training and testing corpora**, construct **unigram, bigram, and trigram models**, estimate probabilities for words in the test sentences, calculate **entropy**, identify sequences with high and low entropy, and compare the effect of smoothing.

Below is a complete Python implementation suitable for the design task.

---

1. Objective

To evaluate an English N-gram language model using **entropy** and compare the uncertainty of unigram, bigram, and trigram models.

The basic entropy formula is:

$$H = -\frac{1}{N}\sum_{i=1}^{N}\log_2 P(w_i|context)$$

where:

- H = entropy
- N = number of predicted words
- P(wi|context) = probability of the word given its context

Interpretation

- **Low entropy** → the next word is more predictable.
- **High entropy** → the next word is less predictable.

---

2. Complete Python Program

```
import re
import math
from collections import Counter



# ============================================================
# TRAINING CORPUS
# ============================================================

training_corpus = """
The student is studying natural language processing.
The student is learning Python programming.
The student is reading an English book.
The student is writing a Python program.
The student is using Python for language processing.
The student is solving a programming problem.
The student is practicing Python programming.
The teacher is teaching the student.
The teacher is explaining natural language processing.
The teacher is reading an English book.
The teacher is giving a programming lesson.
The programmer is writing a Python program.
The programmer is learning natural language processing.
The programmer is developing a language model.
The language model is predicting the next word.
The language model is learning from text.
The language model is processing English text.
"""
```

```
# ============================================================

# SEPARATE TEST CORPUS

# ============================================================


test_corpus = """

The student is learning Python programming.

The teacher is reading an English book.

The programmer is developing a language model.

The language model is predicting the next word.

"""




# ============================================================

# PREPROCESSING

# ============================================================


def preprocess(text):


    text = text.lower()


    sentences = re.split(r'[.!?]+', text)


    processed_sentences = []


    for sentence in sentences:
```

```
        words = re.findall(
            r'\b[a-z]+\b',
            sentence
        )

        if words:
            words = (
                ['<START>']
                + words
                + ['<END>']
            )

            processed_sentences.append(words)

    return processed_sentences


# ========================================================
# BUILD N-GRAM COUNTS
# ========================================================

def build_ngrams(sentences):

    unigram = Counter()
    bigram = Counter()
    trigram = Counter()

    for sentence in sentences:
```

```
        # Unigram
        for word in sentence:
            unigram[word] += 1

        # Bigram
        for i in range(len(sentence) - 1):

            bigram[
                (sentence[i], sentence[i + 1])
            ] += 1

        # Trigram
        for i in range(len(sentence) - 2):

            trigram[
                (
                    sentence[i],
                    sentence[i + 1],
                    sentence[i + 2]
                )
            ] += 1

    return unigram, bigram, trigram


# ============================================================
# PROBABILITY FUNCTIONS
```

```
# ============================================================


def unigram_probability(word, unigram):

    total = sum(unigram.values())

    if total == 0:
        return 0

    return unigram[word] / total



def bigram_probability(
        word1,
        word2,
        unigram,
        bigram):

    denominator = unigram[word1]

    if denominator == 0:
        return 0

    return (
        bigram[(word1, word2)]
        / denominator
    )
```

```
def trigram_probability(
        word1,
        word2,
        word3,
        bigram,
        trigram):

    denominator = bigram[
        (word1, word2)
    ]

    if denominator == 0:
        return 0

    return (
        trigram[
            (word1, word2, word3)
        ]
        / denominator
    )



# ========================================================
# LAPLACE SMOOTHING
# ========================================================

def smoothed_unigram_probability(
```

```
        word,
        unigram,
        vocabulary_size):

    total = sum(unigram.values())

    return (
        unigram[word] + 1
    ) / (
        total + vocabulary_size
    )


def smoothed_bigram_probability(
        word1,
        word2,
        unigram,
        bigram,
        vocabulary_size):

    numerator = (
        bigram[(word1, word2)] + 1
    )

    denominator = (
        unigram[word1]
        + vocabulary_size
    )
```

```
    return numerator / denominator



def smoothed_trigram_probability(
        word1,
        word2,
        word3,
        bigram,
        trigram,
        vocabulary_size):

    numerator = (
        trigram[
            (word1, word2, word3)
        ] + 1
    )

    denominator = (
        bigram[(word1, word2)]
        + vocabulary_size
    )

    return numerator / denominator



# ========================================================
# ENTROPY CALCULATION
```

```
# ========================================================


def calculate_entropy(probabilities):


    if len(probabilities) == 0:
        return float('inf')


    total_log_probability = 0


    for probability in probabilities:


        if probability <= 0:
            return float('inf')


        total_log_probability += (
            math.log2(probability)
        )


    entropy = (
        -total_log_probability
        / len(probabilities)
    )


    return entropy



# ========================================================

# EVALUATE UNIGRAM MODEL
```

```
# ========================================================


def evaluate_unigram(
        test_sentences,
        unigram):

    probabilities = []

    for sentence in test_sentences:

        for word in sentence:

            if word == '<START>':
                continue

            probability = unigram_probability(
                word,
                unigram
            )

            probabilities.append(
                probability
            )

    return calculate_entropy(
        probabilities
    )
```

```
# ============================================================

# EVALUATE BIGRAM MODEL

# ============================================================


def evaluate_bigram(
        test_sentences,
        unigram,
        bigram):


    probabilities = []


    for sentence in test_sentences:


        for i in range(1, len(sentence)):


            previous_word = sentence[i - 1]
            current_word = sentence[i]


            probability = bigram_probability(
                previous_word,
                current_word,
                unigram,
                bigram
            )


            probabilities.append(
                probability
```

```
        )

    return calculate_entropy(
        probabilities
    )



# ========================================================
# EVALUATE TRIGRAM MODEL
# ========================================================

def evaluate_trigram(
        test_sentences,
        bigram,
        trigram):

    probabilities = []

    for sentence in test_sentences:

        for i in range(2, len(sentence)):

            word1 = sentence[i - 2]
            word2 = sentence[i - 1]
            word3 = sentence[i]

            probability = trigram_probability(
                word1,
```

```
            word2,
            word3,
            bigram,
            trigram
        )

        probabilities.append(
            probability
        )

    return calculate_entropy(
        probabilities
    )



# ============================================================
# SMOOTHED TRIGRAM ENTROPY
# ============================================================

def evaluate_smoothed_trigram(
        test_sentences,
        bigram,
        trigram,
        vocabulary_size):

    probabilities = []

    for sentence in test_sentences:
```

```
    for i in range(2, len(sentence)):

        word1 = sentence[i - 2]
        word2 = sentence[i - 1]
        word3 = sentence[i]

        probability = (
            smoothed_trigram_probability(
                word1,
                word2,
                word3,
                bigram,
                trigram,
                vocabulary_size
            )
        )

        probabilities.append(
            probability
        )

    return calculate_entropy(
        probabilities
    )


# ==================================================
```

```
# SENTENCE ENTROPY
# =====================================================

def sentence_trigram_entropy(
        sentence,
        bigram,
        trigram):

    probabilities = []

    for i in range(2, len(sentence)):

        w1 = sentence[i - 2]
        w2 = sentence[i - 1]
        w3 = sentence[i]

        probability = trigram_probability(
            w1,
            w2,
            w3,
            bigram,
            trigram
        )

        probabilities.append(
            probability
        )
```

```
    return calculate_entropy(
        probabilities
    )



# ============================================================
# MAIN PROGRAM
# ============================================================

train_sentences = preprocess(
    training_corpus
)

test_sentences = preprocess(
    test_corpus
)



# Build models

unigram, bigram, trigram = build_ngrams(
    train_sentences
)



vocabulary = set(unigram.keys())

vocabulary_size = len(vocabulary)
```

```
# ============================================================
# DISPLAY INFORMATION
# ============================================================

print("================================================")
print(" ENTROPY-BASED N-GRAM LANGUAGE MODEL")
print("================================================")

print(
    "\nTraining sentences:",
    len(train_sentences)
)

print(
    "Testing sentences:",
    len(test_sentences)
)

print(
    "Vocabulary size:",
    vocabulary_size
)

print(
    "Unique unigrams:",
    len(unigram)
```

```
)

print(
    "Unique bigrams:",
    len(bigram)
)

print(
    "Unique trigrams:",
    len(trigram)
)


# ========================================================
# CALCULATE ENTROPY
# ========================================================

unigram_entropy = evaluate_unigram(
    test_sentences,
    unigram
)

bigram_entropy = evaluate_bigram(
    test_sentences,
    unigram,
    bigram
)
```

```
trigram_entropy = evaluate_trigram(
    test_sentences,
    bigram,
    trigram
)

smoothed_trigram_entropy = (
    evaluate_smoothed_trigram(
        test_sentences,
        bigram,
        trigram,
        vocabulary_size
    )
)



# ========================================================
# DISPLAY ENTROPY
# ========================================================

print("\n================================================")
print(" ENTROPY RESULTS")
print("================================================")

print(
    f"Unigram Entropy : "
    f"{unigram_entropy:.4f}"
)
```

```
print(
    f"Bigram Entropy  : "
    f"{bigram_entropy:.4f}"
)

print(
    f"Trigram Entropy : "
    f"{trigram_entropy:.4f}"
)

print(
    f"Smoothed Trigram Entropy : "
    f"{smoothed_trigram_entropy:.4f}"
)



# ============================================================
# SENTENCE LEVEL ANALYSIS
# ============================================================

print("\n================================================")
print(" SENTENCE-LEVEL ENTROPY")
print("================================================")



sentence_results = []
```

```
for sentence in test_sentences:


    entropy = sentence_trigram_entropy(
        sentence,
        bigram,
        trigram
    )


    sentence_text = " ".join(
        word
        for word in sentence
        if word not in [
            "<START>",
            "<END>"
        ]
    )


    sentence_results.append(
        (
            sentence_text,
            entropy
        )
    )


    print(
        f"\nSentence: {sentence_text}"
    )
```

```
    if entropy == float('inf'):

        print(
            "Entropy: Infinity "
            "(unseen trigram)"
        )

    else:

        print(
            f"Entropy: {entropy:.4f}"
        )



# ============================================================
# HIGH / LOW ENTROPY
# ============================================================

finite_results = [
    item
    for item in sentence_results
    if item[1] != float('inf')
]



print("\n============================================")
print(" HIGH / LOW ENTROPY ANALYSIS")
print("============================================")
```

```
if finite_results:

    lowest = min(
        finite_results,
        key=lambda x: x[1]
    )

    highest = max(
        finite_results,
        key=lambda x: x[1]
    )

    print(
        "\nLowest entropy sentence:"
    )

    print(
        lowest[0]
    )

    print(
        f"Entropy = {lowest[1]:.4f}"
    )

    print(
        "\nHighest finite entropy sentence:"
```

```
    )

    print(
        highest[0]
    )

    print(
        f"Entropy = {highest[1]:.4f}"
    )



# ============================================================
# INTERPRETATION
# ============================================================

print("\n================================================")
print(" INTERPRETATION")
print("================================================")

print("""
Low entropy means that the next words are relatively
predictable from their context.

High entropy means that the model has greater uncertainty
about the next word.

The trigram model uses more context than the unigram and
bigram models.
```

Smoothing assigns non-zero probabilities to unseen N-grams,

which makes entropy calculation possible for test sequences

containing unseen combinations.

""")

---

3. Important Concept: Entropy

Entropy measures **uncertainty** in the language model.

For a sequence of words:

$H(W)=-N1i=1\sum Nlog2P(wi|context)$

For example, suppose the model predicts:

P(programming | Python) = 0.8

This is highly predictable, so its contribution to entropy is relatively low.

If:

P(elephant | Python) = 0.001

the model considers it very unlikely, resulting in a much larger contribution to entropy.

Therefore:

Higher probability → lower uncertainty → lower entropy

Lower probability → higher uncertainty → higher entropy

---

4. Training and Testing Corpus

The program deliberately keeps the corpora separate:

Training corpus

↓

Build N-gram models

↓

Calculate probabilities

↓

Testing corpus

↓

Evaluate probabilities

↓

Calculate entropy

This is important because the test corpus should be used for **evaluation**, rather than simply being counted as part of the training data.

The assignment specifically requires a **training corpus and a separate test corpus**.

---

5. Unigram, Bigram and Trigram

Unigram

Uses no previous context:

$P(w)=\text{Total WordsCount}(w)$

Example:

P(student)

Bigram

Uses one previous word:

$P(wi|wi-1)=\text{Count}(wi-1)\text{Count}(wi-1,wi)$

Example:

P(student | the)

Trigram

Uses two previous words:

$P(wi|wi-2,wi-1)=\text{Count}(wi-2,wi-1)\text{Count}(wi-2,wi-1,wi)$

Example:

P(programming | Python, student)

---

6. Why Unsmoothed Entropy Can Become Infinity

This is a very important point for your viva.

Suppose the test corpus contains a trigram that never occurred in the training corpus:

student is elephant

Then:

$P(\text{elephant}|\text{student,is})=0$

The entropy calculation contains:

$-\log_2(0)$

which tends toward:

∞

Therefore, the program reports:

Entropy: Infinity (unseen trigram)

This demonstrates why smoothing is important when evaluating a language model on unseen test data.

---

7. Effect of Smoothing

The program also implements **Laplace/add-one smoothing** for the trigram model:

$P_{smooth}=\text{Count}(w_{i-2},w_{i-1})+V\text{Count}(w_{i-2},w_{i-1},w_i)+1$

where V is the vocabulary size.

Thus, even if:

$\text{Count}(w_1,w_2,w_3) = 0$

the smoothed probability is not zero:

$P_{smooth}>0$

Consequently, the entropy calculation can proceed without log(0).

The assignment specifically asks you to **compare the effect of smoothing on entropy** and explain why better probability estimation produces more reliable predictions.

---

8. Sample Output Format

When executed, the program produces output similar to:

==========

ENTROPY-BASED N-GRAM LANGUAGE MODEL

==========

Training sentences: 17

Testing sentences: 4

Vocabulary size: ...

Unique unigrams: ...

Unique bigrams: ...

Unique trigrams: ...

==========

ENTROPY RESULTS

==========

Unigram Entropy : ...

Bigram Entropy : ...

Trigram Entropy : ...

Smoothed Trigram Entropy : ...

==========

SENTENCE-LEVEL ENTROPY

==========

Sentence: the student is learning python programming

Entropy: ...

Sentence: the teacher is reading an english book

Entropy: ...

Sentence: the programmer is developing a language model

Entropy: ...

Sentence: the language model is predicting the next word

Entropy: ...

Don't copy numerical values from this example—run the program and use the values printed by your execution, because the exact values depend on the corpus and implementation.

---

9. High- and Low-Entropy Sequences

The program identifies the sentences with the lowest and highest **finite** trigram entropy.

Low entropy

A sentence has low entropy when its word sequences are common in the training corpus.

For example:

The language model is predicting the next word.

If this sequence occurs frequently in training, the model is confident about the next words.

High entropy

A sentence has high entropy when its word sequences are rare or unseen.

For example:

The student is ...

may have several possible continuations, producing greater uncertainty.

An entirely unseen trigram produces infinite unsmoothed entropy, which is why the program separately identifies Infinity.

---

10. Comparison

Model	Context	Unseen N-gram	Entropy
Unigram	No context	Can be 0	Measures overall word uncertainty
Bigram	Previous word	Can be 0	Uses local context
Trigram	Previous 2 words	Can be 0	Uses more context
Smoothed Trigram	Previous 2 words	Non-zero probability	More robust evaluation

A key observation is that **more context does not automatically guarantee a lower measured entropy on every test corpus**. A trigram model can encounter many unseen sequences in a small training corpus. Smoothing addresses this sparsity problem.

---

11. Advantages

- Provides a quantitative measure of language-model uncertainty.
- Allows comparison between different N-gram orders.
- Separates training and testing data.
- Identifies predictable and unpredictable sequences.
- Demonstrates the effect of smoothing.
- Useful for evaluating text-prediction systems.

12. Limitations

- Results depend strongly on corpus size and quality.
- Small corpora produce many unseen N-grams.
- N-gram models have limited context.
- Entropy alone does not measure semantic correctness.
- Unsmoothed models can produce infinite entropy for unseen sequences.
- Laplace smoothing is simple but may not be the best smoothing technique for a real-world language model.

---

13. Result

Result:
A Python-based entropy evaluation system for an English N-gram language model was successfully designed. Separate training and testing corpora were used to construct unigram, bigram, and trigram models and calculate word probabilities. Entropy was computed to measure uncertainty, high- and low-entropy sequences were identified, and the effect of smoothing on unseen N-grams was demonstrated. This fulfills the Q3 requirements specified in the assessment document.

```
==========================================
 ENTROPY-BASED N-GRAM LANGUAGE MODEL
==========================================

Training sentences: 17
Testing sentences: 4
Vocabulary size: 37
Unique unigrams: 37
Unique bigrams: 65
Unique trigrams: 76

==========================================
 ENTROPY RESULTS
==========================================
Unigram Entropy : 4.6436
Bigram Entropy  : 1.1955
Trigram Entropy : 0.7337
Smoothed Trigram Entropy : 3.8519

==========================================
 SENTENCE-LEVEL ENTROPY
==========================================

Sentence: the student is learning python programming
Entropy: 0.9775

Sentence: the teacher is reading an english book
Entropy: 0.5839

Sentence: the programmer is developing a language model
Entropy: 0.8696

Sentence: the language model is predicting the next word
Entropy: 0.5628

==========================================
 HIGH / LOW ENTROPY ANALYSIS
==========================================

Lowest entropy sentence:
the language model is predicting the next word
Entropy = 0.5628

Highest finite entropy sentence:
the student is learning python programming
Entropy = 0.9775

==========================================
 INTERPRETATION
==========================================

Low entropy means that the next words are relatively
predictable from their context.

High entropy means that the model has greater uncertainty
about the next word.

The trigram model uses more context than the unigram and
bigram models.

Smoothing assigns non-zero probabilities to unseen N-grams,
which makes entropy calculation possible for test sequences
containing unseen combinations.

PS D:\NLP\Assessments\CO3 Assessment 1>
```

Q4 — Part-of-Speech Tagging System

The assignment asks you to implement three approaches for POS tagging:

1. **Rule-Based POS Tagger** using lexical dictionaries and grammatical rules.
2. **Stochastic POS Tagger** using word/tag probabilities and transition probabilities.
3. **Transformation-Based Tagging** where initially assigned tags are corrected using transformation rules.

The final system should compare the three approaches and use an English tagset such as the **Penn Treebank tagset**.

---

1. Objective

To design a Python-based POS tagging system that assigns grammatical tags such as:

Noun → NN

Verb → VB

Adjective → JJ

Adverb → RB

Pronoun → PRP

Preposition → IN

Conjunction → CC

and compare:

Rule-Based

↓

Stochastic

↓

Transformation-Based

---

2. Complete Python Program

This version is **self-contained** and does not require downloading NLTK data.

```
import re

import math
```

```
from collections import Counter, defaultdict



# ============================================================

# PENN TREEBANK TAGSET - BASIC TAGS

# ============================================================


TAG_DESCRIPTIONS = {

    "NN": "Noun, singular",

    "NNS": "Noun, plural",

    "VB": "Verb, base form",

    "VBD": "Verb, past tense",

    "VBG": "Verb, gerund/present participle",

    "VBN": "Verb, past participle",

    "VBZ": "Verb, 3rd person singular",

    "JJ": "Adjective",

    "RB": "Adverb",

    "PRP": "Personal pronoun",

    "DT": "Determiner",

    "IN": "Preposition/subordinating conjunction",

    "CC": "Coordinating conjunction",

    "TO": "to",

    "MD": "Modal",

    "CD": "Cardinal number",

}



# ============================================================
```

```
# TRAINING DATA
# ============================================================
#
# Format:
# sentence = [(word, correct_tag), ...]
#
# This small tagged corpus is used to calculate
# lexical and transition probabilities.
# ============================================================

training_data = [

    [
        ("the", "DT"),
        ("student", "NN"),
        ("is", "VBZ"),
        ("reading", "VBG"),
        ("a", "DT"),
        ("book", "NN")
    ],

    [
        ("the", "DT"),
        ("student", "NN"),
        ("is", "VBZ"),
        ("writing", "VBG"),
        ("a", "DT"),
        ("program", "NN")
```

```
],

[
    ("the", "DT"),
    ("teacher", "NN"),
    ("is", "VBZ"),
    ("teaching", "VBG"),
    ("the", "DT"),
    ("student", "NN")
],

[
    ("the", "DT"),
    ("programmer", "NN"),
    ("is", "VBZ"),
    ("writing", "VBG"),
    ("python", "NN")
],

[
    ("the", "DT"),
    ("student", "NN"),
    ("learns", "VBZ"),
    ("python", "NN"),
    ("programming", "NN")
],

[
```

```
    ("the", "DT"),
    ("teacher", "NN"),
    ("explains", "VBZ"),
    ("natural", "JJ"),
    ("language", "NN")
],

[
    ("the", "DT"),
    ("student", "NN"),
    ("quickly", "RB"),
    ("reads", "VBZ"),
    ("the", "DT"),
    ("book", "NN")
],

[
    ("the", "DT"),
    ("programmer", "NN"),
    ("carefully", "RB"),
    ("writes", "VBZ"),
    ("code", "NN")
],

[
    ("i", "PRP"),
    ("am", "VBP"),
    ("learning", "VBG"),
```

```
    ("python", "NN")
],

[
    ("we", "PRP"),
    ("are", "VBP"),
    ("studying", "VBG"),
    ("natural", "JJ"),
    ("language", "NN"),
    ("processing", "NN")
],

[
    ("the", "DT"),
    ("student", "NN"),
    ("can", "MD"),
    ("write", "VB"),
    ("python", "NN")
],

[
    ("the", "DT"),
    ("student", "NN"),
    ("will", "MD"),
    ("learn", "VB"),
    ("programming", "NN")
],
```

```
    [
        ("the", "DT"),
        ("teacher", "NN"),
        ("and", "CC"),
        ("student", "NN"),
        ("read", "VB"),
        ("together", "RB")
    ],

    [
        ("the", "DT"),
        ("programmer", "NN"),
        ("uses", "VBZ"),
        ("python", "NN")
    ]
]



# ============================================================
# PREPROCESSING
# ============================================================

def tokenize(sentence):

    return re.findall(
        r"\b[a-zA-Z]+\b",
        sentence.lower()
    )
```

```
# ============================================================

# BUILD STATISTICS

# ============================================================


def build_statistics(training_data):


    word_tag_counts = defaultdict(Counter)

    tag_counts = Counter()

    transition_counts = defaultdict(Counter)


    vocabulary = set()


    for sentence in training_data:


        previous_tag = "<START>"


        for word, tag in sentence:


            vocabulary.add(word)


            word_tag_counts[word][tag] += 1


            tag_counts[tag] += 1


            transition_counts[

                previous_tag
```

```
            ][tag] += 1

            previous_tag = tag

        transition_counts[
            previous_tag
        ]["<END>"] += 1

    return (
        word_tag_counts,
        tag_counts,
        transition_counts,
        vocabulary
    )


word_tag_counts, tag_counts, transition_counts, vocabulary = \
    build_statistics(training_data)


# ======================================================
# LEXICAL DICTIONARIES FOR RULE-BASED TAGGING
# ======================================================

LEXICON = {

    "the": "DT",
    "a": "DT",
```

"an": "DT",

"i": "PRP",

"we": "PRP",

"you": "PRP",

"he": "PRP",

"she": "PRP",

"they": "PRP",

"is": "VBZ",

"are": "VBP",

"am": "VBP",

"can": "MD",

"will": "MD",

"should": "MD",

"must": "MD",

"and": "CC",

"or": "CC",

"but": "CC",

"to": "TO",

"in": "IN",

"on": "IN",

"at": "IN",

"for": "IN",

```
    "with": "IN",
    "from": "IN",

    "student": "NN",
    "teacher": "NN",
    "programmer": "NN",
    "book": "NN",
    "program": "NN",
    "python": "NN",
    "language": "NN",
    "processing": "NN",
    "code": "NN",
    "programming": "NN",

    "natural": "JJ",

    "quickly": "RB",
    "carefully": "RB",
    "together": "RB"
}



# ==========================================================
# RULE-BASED POS TAGGER
# ==========================================================

def rule_based_tag(sentence):
```

```
words = tokenize(sentence)

tagged = []

previous_word = None

for word in words:

    # ----------------------------------------------
    # Rule 1: Lexical dictionary
    # ----------------------------------------------

    if word in LEXICON:

        tag = LEXICON[word]

    # ----------------------------------------------
    # Rule 2: Number
    # ----------------------------------------------

    elif word.isdigit():

        tag = "CD"

    # ----------------------------------------------
    # Rule 3: -ly → adverb
    # ----------------------------------------------
```

```
elif word.endswith("ly"):

    tag = "RB"

# ---------------------------------------------
# Rule 4: -ing → gerund
# ---------------------------------------------

elif word.endswith("ing"):

    tag = "VBG"

# ---------------------------------------------
# Rule 5: -ed → past tense
# ---------------------------------------------

elif word.endswith("ed"):

    tag = "VBD"

# ---------------------------------------------
# Rule 6: -ous, -ful, -able → adjective
# ---------------------------------------------

elif (
    word.endswith("ous")
    or word.endswith("ful")
    or word.endswith("able")
```

```
):

    tag = "JJ"

# ------------------------------------------------
# Rule 7: After modal → base verb
# ------------------------------------------------

elif previous_word in {
    "can",
    "will",
    "should",
    "must"
}:

    tag = "VB"

# ------------------------------------------------
# Rule 8: Default unknown word → noun
# ------------------------------------------------

else:

    tag = "NN"

tagged.append(
    (word, tag)
)
```

```
        previous_word = word

    return tagged


# ============================================================
# STOCHASTIC POS TAGGER
# ============================================================

def emission_probability(word, tag):

    total = tag_counts[tag]

    if total == 0:
        return 0

    return (
        word_tag_counts[word][tag]
        / total
    )


def transition_probability(
        previous_tag,
        current_tag):

    total = sum(
```

```
        transition_counts[
            previous_tag
        ].values()
    )

    if total == 0:
        return 0

    return (
        transition_counts[
            previous_tag
        ][current_tag]
        / total
    )


def stochastic_tag(sentence):

    words = tokenize(sentence)

    all_tags = list(
        tag_counts.keys()
    )

    result = []

    previous_tag = "<START>"
```

```
for word in words:

    best_tag = None
    best_score = -1

    for tag in all_tags:

        # ----------------------------------------
        # Emission probability
        # ----------------------------------------

        emission = (
            word_tag_counts[word][tag]
            + 1
        ) / (
            tag_counts[tag]
            + len(vocabulary)
        )

        # ----------------------------------------
        # Transition probability
        # ----------------------------------------

        transition = (
            transition_counts[
                previous_tag
            ][tag]
            + 1
```

```
    ) / (
        sum(
            transition_counts[
                previous_tag
            ].values()
        )
        + len(all_tags)
    )

    # ----------------------------------------
    # Combined probability
    # ----------------------------------------

    score = (
        math.log(emission)
        + math.log(transition)
    )

    if score > best_score:

        best_score = score
        best_tag = tag

result.append(
    (word, best_tag)
)

previous_tag = best_tag
```

```
    return result


# ============================================================
# TRANSFORMATION-BASED TAGGER
# ============================================================

def transformation_based_tag(sentence):

    # Start with rule-based tags
    tagged = rule_based_tag(sentence)

    # ------------------------------------------------
    # Transformation Rule 1:
    #
    # After a pronoun or modal, an unknown noun
    # can become a verb.
    # ------------------------------------------------

    for i in range(1, len(tagged)):

        word, tag = tagged[i]

        previous_word, previous_tag = \
            tagged[i - 1]

        if (
```

```
        previous_tag == "PRP"
        and tag == "NN"
    ):

        tagged[i] = (
            word,
            "VB"
        )

    elif (
        previous_tag == "MD"
        and tag == "NN"
    ):

        tagged[i] = (
            word,
            "VB"
        )

# ------------------------------------------------
# Transformation Rule 2:
#
# Words ending in -ing after an auxiliary
# are VBG.
# ------------------------------------------------

for i in range(len(tagged)):
```

```
    word, tag = tagged[i]


    if (
        word.endswith("ing")
        and i > 0
    ):


        previous_word, previous_tag = \
            tagged[i - 1]


        if previous_tag in {
            "VBZ",
            "VBP",
            "VBD"
        }:


            tagged[i] = (
                word,
                "VBG"
            )


# ---------------------------------------------
# Transformation Rule 3:
#
# Words ending in -ly are adverbs.
# ---------------------------------------------


for i in range(len(tagged)):
```

```
        word, tag = tagged[i]

        if word.endswith("ly"):

            tagged[i] = (
                word,
                "RB"
            )

    return tagged


# ============================================================
# DISPLAY TAGGED SENTENCE
# ============================================================

def display_result(title, tagged):

    print("\n" + "=" * 60)

    print(title)

    print("=" * 60)

    for word, tag in tagged:

        print(
```

```
            f"{word:15} -> "
            f"{tag:5} "
            f"{TAG_DESCRIPTIONS.get(tag, '')}"
        )


# ============================================================
# COMPARE THREE TAGGERS
# ============================================================

def compare_taggers(sentence):

    print("\n\n")
    print("#" * 60)
    print("INPUT SENTENCE")
    print("#" * 60)

    print(sentence)

    rule_result = rule_based_tag(
        sentence
    )

    stochastic_result = stochastic_tag(
        sentence
    )

    transformation_result = \
```

```
        transformation_based_tag(
            sentence
        )

    display_result(
        "1. RULE-BASED POS TAGGER",
        rule_result
    )

    display_result(
        "2. STOCHASTIC POS TAGGER",
        stochastic_result
    )

    display_result(
        "3. TRANSFORMATION-BASED POS TAGGER",
        transformation_result
    )



# ============================================================
# EVALUATION AGAINST TRAINING DATA
# ============================================================

def calculate_accuracy(
        predicted,
        actual):
```

```
    correct = 0
    total = 0

    for predicted_item, actual_item in zip(
        predicted,
        actual
    ):

        if predicted_item[1] == actual_item[1]:

            correct += 1

        total += 1

    if total == 0:
        return 0

    return (
        correct / total
    ) * 100



def evaluate_systems():

    rule_correct = 0
    stochastic_correct = 0
    transformation_correct = 0
```

```
total = 0


for sentence in training_data:


    text = " ".join(
        word
        for word, tag in sentence
    )


    actual = sentence


    rule_prediction = \
        rule_based_tag(text)


    stochastic_prediction = \
        stochastic_tag(text)


    transformation_prediction = \
        transformation_based_tag(text)


    for i in range(len(actual)):


        total += 1


        if (
            rule_prediction[i][1]
            == actual[i][1]
        ):
```

```
            rule_correct += 1

        if (
            stochastic_prediction[i][1]
            == actual[i][1]
        ):

            stochastic_correct += 1

        if (
            transformation_prediction[i][1]
            == actual[i][1]
        ):

            transformation_correct += 1

print("\n")
print("=" * 60)
print("POS TAGGER EVALUATION")
print("=" * 60)

print(
    f"Rule-Based Accuracy          : "
    f"{rule_correct / total * 100:.2f}%"
)

print(
```

```
        f"Stochastic Accuracy          : "
        f"{stochastic_correct / total * 100:.2f}%"
    )

    print(
        f"Transformation-Based Accuracy: "
        f"{transformation_correct / total * 100:.2f}%"
    )


# ============================================================
# MAIN PROGRAM
# ============================================================

print("=" * 60)
print("      PART-OF-SPEECH TAGGING SYSTEM")
print("=" * 60)

while True:

    print("\n")
    print("1. Rule-Based POS Tagging")
    print("2. Stochastic POS Tagging")
    print("3. Transformation-Based Tagging")
    print("4. Compare All Three")
    print("5. Evaluate Taggers")
    print("6. Display Tagset")
    print("7. Exit")
```

```
choice = input(
    "\nEnter your choice: "
)

if choice == "1":

    sentence = input(
        "Enter an English sentence: "
    )

    result = rule_based_tag(
        sentence
    )

    display_result(
        "RULE-BASED POS TAGGING",
        result
    )

elif choice == "2":

    sentence = input(
        "Enter an English sentence: "
    )

    result = stochastic_tag(
        sentence
```

```
    )

    display_result(
        "STOCHASTIC POS TAGGING",
        result
    )

elif choice == "3":

    sentence = input(
        "Enter an English sentence: "
    )

    result = transformation_based_tag(
        sentence
    )

    display_result(
        "TRANSFORMATION-BASED POS TAGGING",
        result
    )

elif choice == "4":

    sentence = input(
        "Enter an English sentence: "
    )
```

```
    compare_taggers(
        sentence
    )

elif choice == "5":

    evaluate_systems()

elif choice == "6":

    print("\nPENN TREEBANK TAGSET")
    print("-" * 60)

    for tag, description in \
            TAG_DESCRIPTIONS.items():

        print(
            f"{tag:5} -> {description}"
        )

elif choice == "7":

    print(
        "\nProgram terminated."
    )

    break
```

```
else:


    print(
        "\nInvalid choice."
    )
```

---

3. How the System Works

A. Rule-Based POS Tagger

The first approach uses a **lexical dictionary plus grammatical rules**, as required by the assignment.

For example:

the → DT

student → NN

is → VBZ

quickly → RB

It also uses suffix rules:

-ing → VBG

-ed → VBD

-ly → RB

and contextual rules such as:

can + write

will + learn

where the word after a modal is assigned VB.

Example

Input:

The student is reading a book.

Possible output:

The → DT

student → NN

is → VBZ

reading → VBG

a → DT

book → NN

---

4. Stochastic POS Tagger

The stochastic model uses:

Emission probability

P(word|tag)

This represents how likely a word is to occur with a particular POS tag.

For example:

P(student|NN)

is high if student frequently occurs as a noun.

Transition probability

P(tagi|tagi−1)

This represents how likely one tag is to follow another.

For example:

P(NN|DT)

can be high because patterns such as:

the student

the teacher

the programmer

are common.

The program combines emission and transition probabilities to select the most likely tag.

---

5. Transformation-Based Tagging

The transformation-based model first generates an initial tagging and then **corrects tags using transformation rules**, matching the approach specified in the assignment.

For example:

Initial:

he → PRP

write → NN

A transformation rule can change:

PRP + NN

into:

PRP + VB

giving:

he → PRP

write → VB

Another rule identifies words ending in:

-ly → RB

and:

-ing → VBG

---

6. Example Comparison

Try:

The student is reading a book.

The output will have the following structure:

==========

1. RULE-BASED POS TAGGER

==========

the -> DT

student -> NN

is -> VBZ

reading -> VBG

a -> DT

book -> NN

The stochastic and transformation-based approaches will then provide their own tag sequences for the same input.

This allows you to directly compare the three methods, as required by Q4.

---

7. Penn Treebank Tags Used

Tag	Meaning	Example
NN	Singular noun	student
NNS	Plural noun	students
VB	Base verb	write
VBD	Past-tense verb	worked
VBG	Gerund/present participle	reading
VBN	Past participle	written
VBZ	3rd-person singular verb	reads
JJ	Adjective	natural
RB	Adverb	quickly
PRP	Personal pronoun	he
DT	Determiner	the
IN	Preposition	in
CC	Conjunction	and
MD	Modal	can
TO	to	to
CD	Cardinal number	10

The assignment specifically asks for an appropriate English tagset such as **Penn Treebank**.

---

8. Test Cases

Test Case 1

The student is reading a book.

Expected important tags:

The → DT

student → NN

is → VBZ

reading → VBG

a → DT

book → NN

Test Case 2

The programmer carefully writes code.

Expected:

The → DT

programmer → NN

carefully → RB

writes → VBZ

code → NN

Test Case 3

I am learning Python.

Expected:

I → PRP

am → VBP

learning → VBG

Python → NN

Test Case 4

The student can write Python.

Expected:

The → DT

student → NN

can → MD

write → VB

Python → NN

---

9. Comparison of the Three Approaches

Feature	Rule-Based	Stochastic	Transformation-Based
Uses dictionary	Yes	Yes, through training statistics	Initial tagging
Uses grammatical rules	Yes	No explicit rules	Yes
Uses probabilities	No	Yes	Not primarily
Uses transition probabilities	No	Yes	No
Corrects initial tags	No	No	Yes
Handles unknown words	Suffix/context rules	Smoothing/default probabilities	Transformation rules
Main advantage	Simple and interpretable	Probability-based	Can correct systematic errors

---

10. Analysis

Rule-Based

Advantages

- Easy to understand.
- Rules are transparent.
- Does not require a large training corpus.

Limitations

- Many exceptions exist in English.
- Rules can become complicated.
- Unknown words are difficult to classify correctly.

Stochastic

Advantages

- Uses statistical evidence.
- Can learn tag transitions from training data.
- Better suited to ambiguous words when sufficient training data exists.

Limitations

- Requires a tagged training corpus.
- Small training data can produce unreliable probabilities.
- Unknown words remain challenging.

Transformation-Based

Advantages

- Combines an initial tagger with correction rules.
- Can fix systematic tagging mistakes.
- Rules are interpretable.

Limitations

- Requires carefully designed transformation rules.
- Performance depends on the initial tagging.
- A small set of rules cannot cover all English grammatical patterns.

---

11. Result

Result:
A Python-based POS tagging system was designed using **Rule-Based, Stochastic, and Transformation-Based** approaches. The system uses lexical and grammatical rules for the rule-based tagger, word/tag and transition probabilities for the stochastic tagger, and transformation rules to correct initial tags. The three approaches can be applied to the same input sentences and compared using the Penn Treebank tagset, fulfilling the requirements specified for Q4.

```
============================================================
       PART-OF-SPEECH TAGGING SYSTEM
============================================================


1. Rule-Based POS Tagging
2. Stochastic POS Tagging
3. Transformation-Based Tagging
4. Compare All Three
5. Evaluate Taggers
6. Display Tagset
7. Exit

Enter your choice: 1
Enter an English sentence: My name is Rohtih

============================================================
RULE-BASED POS TAGGING
============================================================
my              -> NN    Noun, singular
name            -> NN    Noun, singular
is              -> VBZ   Verb, 3rd person singular
rohtih          -> NN    Noun, singular


1. Rule-Based POS Tagging
2. Stochastic POS Tagging
3. Transformation-Based Tagging
4. Compare All Three
5. Evaluate Taggers
6. Display Tagset
7. Exit

Enter your choice:
```